

PCS-302 : Data Structures Lab

Reverse a String Using a Stack | With and Without Recursion

Problem Statement

Given an empty stack, design an algorithm and a program to reverse a string using this stack (with and without recursion).

Example

Input string: HELLO

Push (left to right): H E L L O --> stack (top to bottom): O L L E H

Pop (in that order): O L L E H

Reversed string: OLLEH

Algorithm — Without Recursion (Explicit Stack)

1. Push each character of the string onto a stack, left to right.
2. Pop characters one by one and place them back into the string (or print them) -- this produces the reversed string.

Algorithm — With Recursion (Call Stack)

```
reverse(str, index, len):
    if index == len:
        return                                // base case: reached the end

    reverse(str, index + 1, len)              // "push" -- go deeper first
    print str[index]                          // "pop" -- print on the way back
```

No array is needed here -- every recursive call that has not returned yet sits on the function call stack, so unwinding those calls naturally pops the characters in reverse order.

C Program — Without Recursion

```
#include <stdio.h>
#include <string.h>
#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    top = top + 1;
    stack[top] = c;
}

char pop() {
    char c = stack[top];
    top = top - 1;
}
```

```

    return c;
}

int main() {
    char str[MAX];
    int i, len;

    printf("Enter a string: ");
    scanf("%s", str);

    len = strlen(str);

    // Push every character onto the stack
    for (i = 0; i < len; i++) {
        push(str[i]);
    }

    // Pop every character back into the string -> reverses it
    for (i = 0; i < len; i++) {
        str[i] = pop();
    }

    printf("Reversed string: %s\n", str);

    return 0;
}

```

C Program — With Recursion

```

#include <stdio.h>
#include <string.h>

void reverseUsingRecursion(char str[], int index, int len) {
    // Base case: reached end of string
    if (index == len) {
        return;
    }

    // Go to the next character first (this "pushes" onto call stack)
    reverseUsingRecursion(str, index + 1, len);

    // Print current character after returning (this "pops")
    printf("%c", str[index]);
}

int main() {
    char str[100];
    int len;

    printf("Enter a string: ");
    scanf("%s", str);

    len = strlen(str);

    printf("Reversed string: ");
}

```

```
reverseUsingRecursion(str, 0, len);  
printf("\n");  
  
return 0;  
}
```